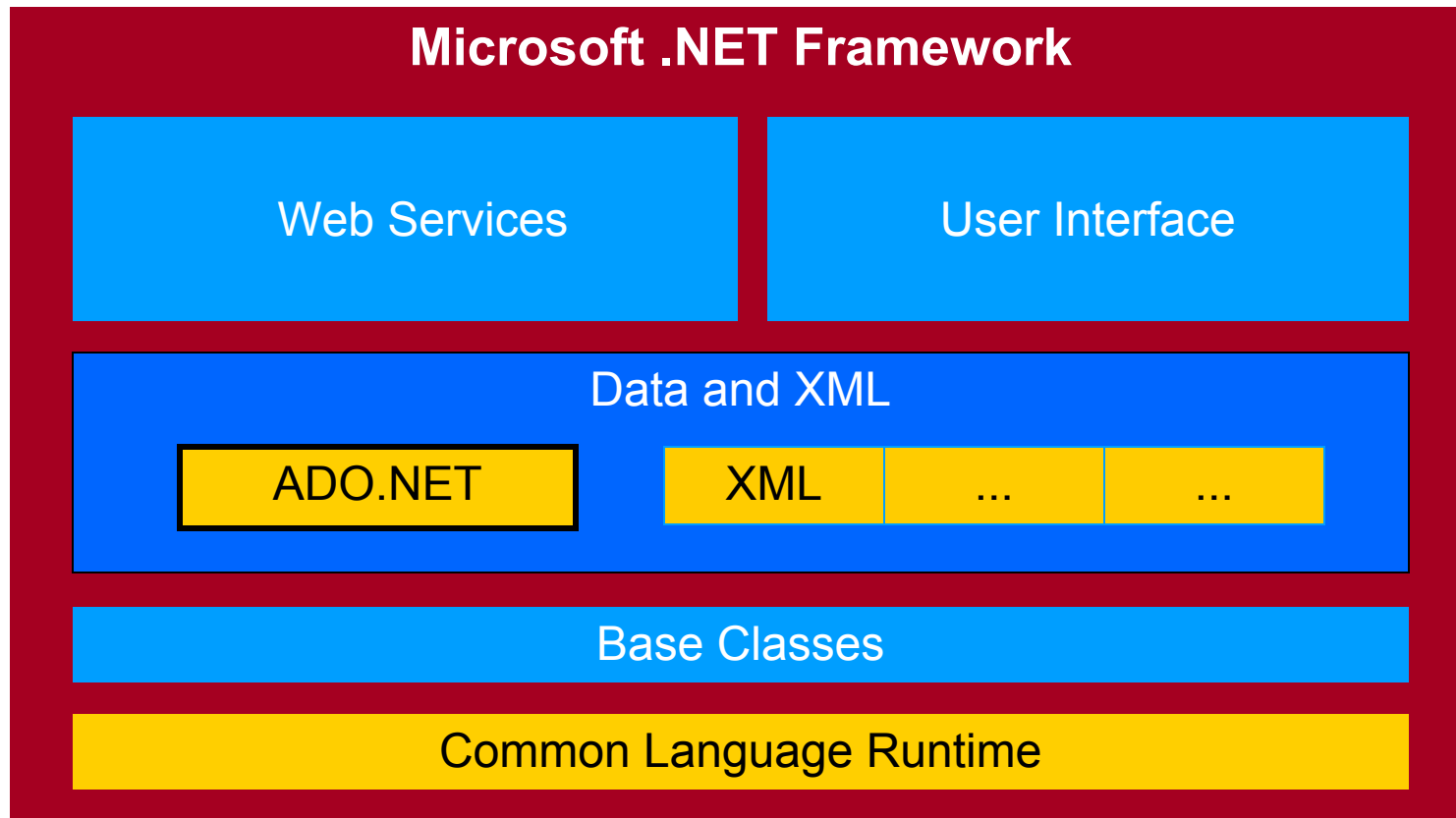


Introduction to ADO.NET

- What we cover...
 - ADO.NET
 - Benefits of ADO.NET
 - ADO.NET Core Concepts and Architecture
 - The ADO.NET Object Model
 - The **DataSet** and Data Views
 - Managed Providers

ADO.NET and the .NET Framework



ADO.NET Overview

What Is ADO.NET?

- ADO .NET is a collection of classes, interfaces, structures, and enumerated types that manage data access from relational data stores within the .NET Framework
 - These collections are organized into namespaces:
 - System.Data, System.Data.OleDb, System.Data.SqlClient, etc.
- ADO .NET is an evolution from ADO.
 - Does not share the same object model, but shares many of the same paradigms and functionality!

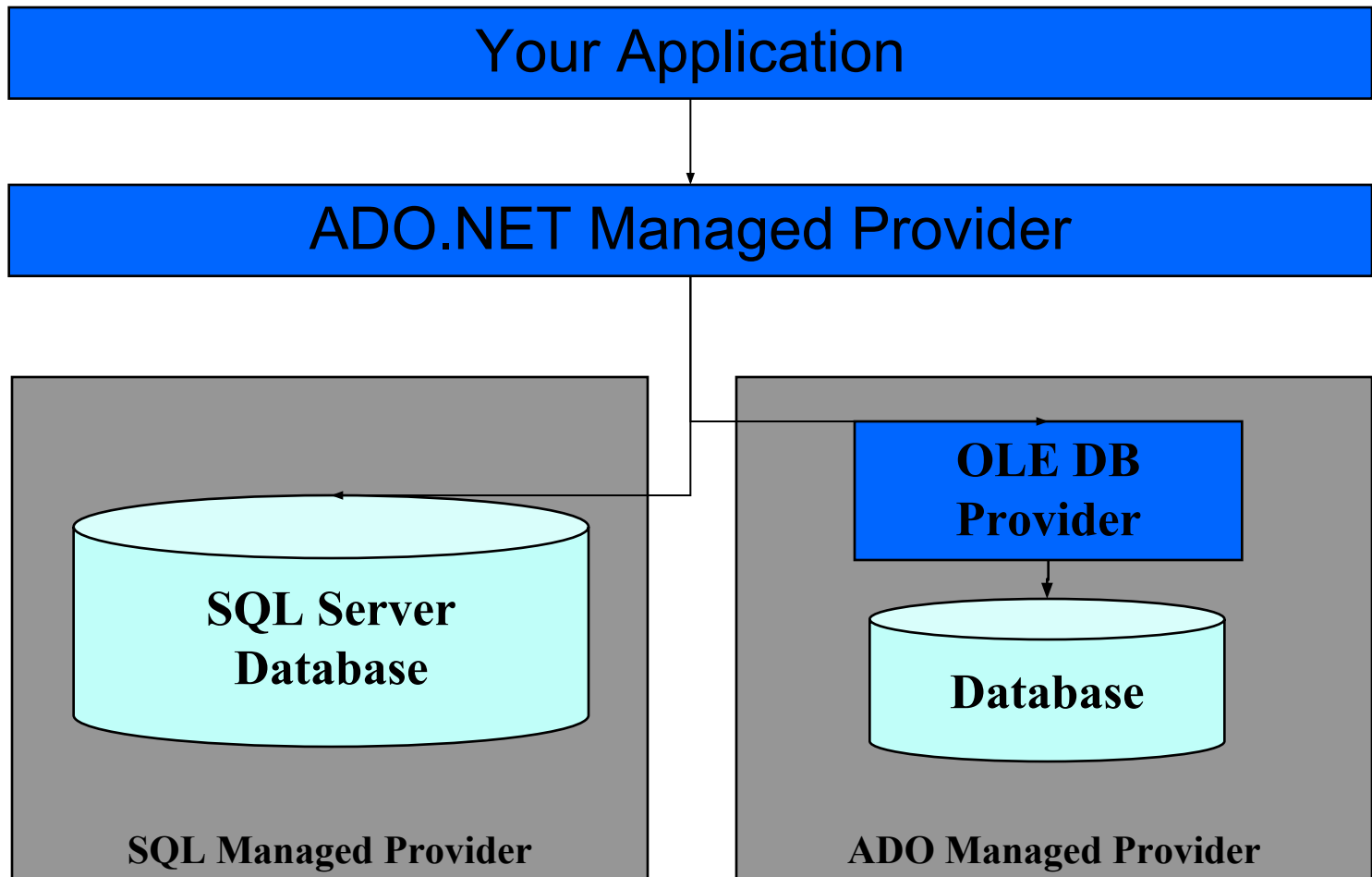
ADO.NET Overview

Managed Providers

- Merges ADO and OLEDB into one layer
- Each provider contains a set of classes that implement common interfaces
- Initial managed provider implementations:
 - ADO Managed Provider: provides access to any OLE DB data source
 - SQL Server Managed Provider: provides optimal performance when using SQL Server
 - Exchange Managed Provider: retrieve and update data in Microsoft Exchange

ADO.NET Overview

Managed Providers



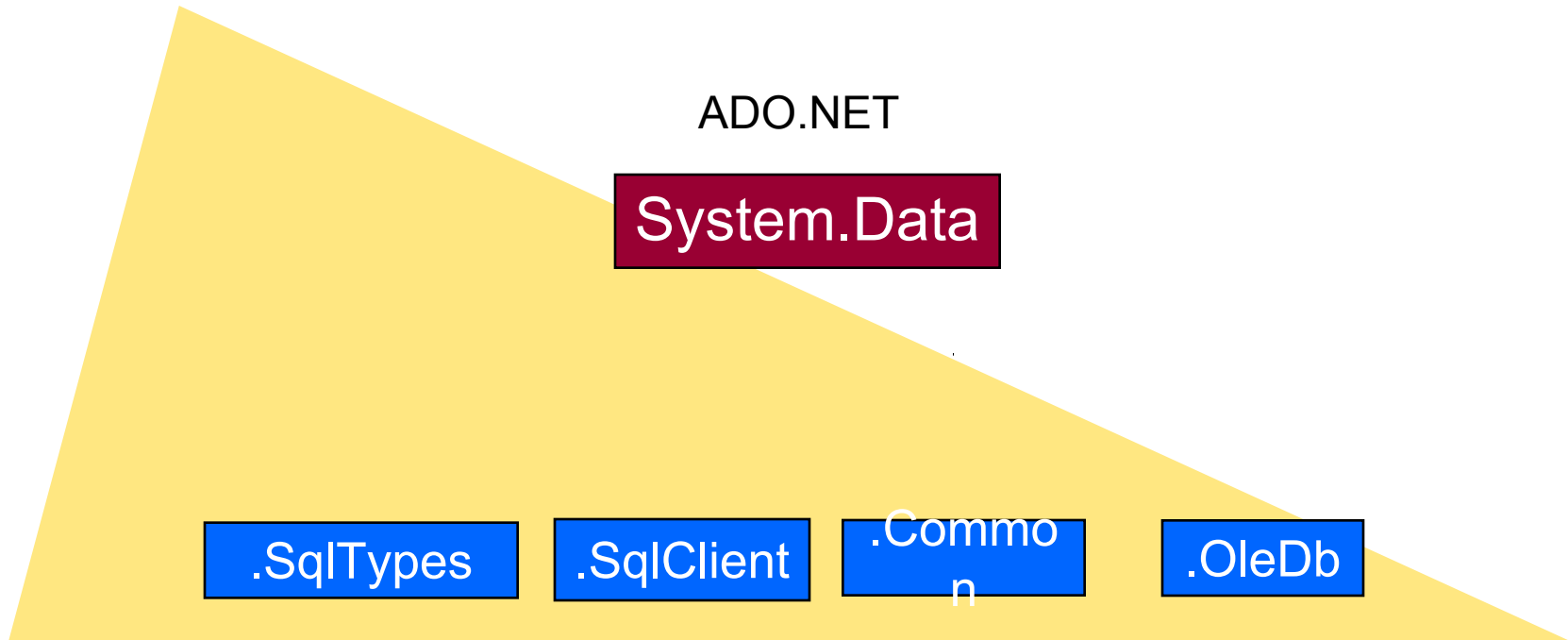
Benefits of ADO.NET

- Interoperability through use of XML
 - Open standard for data that describes itself
 - Human readable and decipherable text
 - Used internally but accessible externally
 - Can use XML to read and write and move data
- Scalability through the disconnected **DataSet**
 - Connections are not maintained for long periods
 - Database locking does not occur
 - Locking support with ServiceComponents
 - Optimistic locking otherwise
 - Works the way the Web works: “Hit and Run!”
- Maintainability
 - Separation of data logic and user interface

Core Concepts and Architecture

- The ADO.NET Object Model
 - Objects of System.Data
 - .NET data providers
- ADO.NET namespace hierarchy
 - Organizes the object model
 - Includes:
 - **System.Data**
 - **System.Data.OleDb**
 - **System.Data.Common**
 - **System.Data.SqlClient**
 - **System.Data.SqlTypes**

ADO.NET-related Namespaces

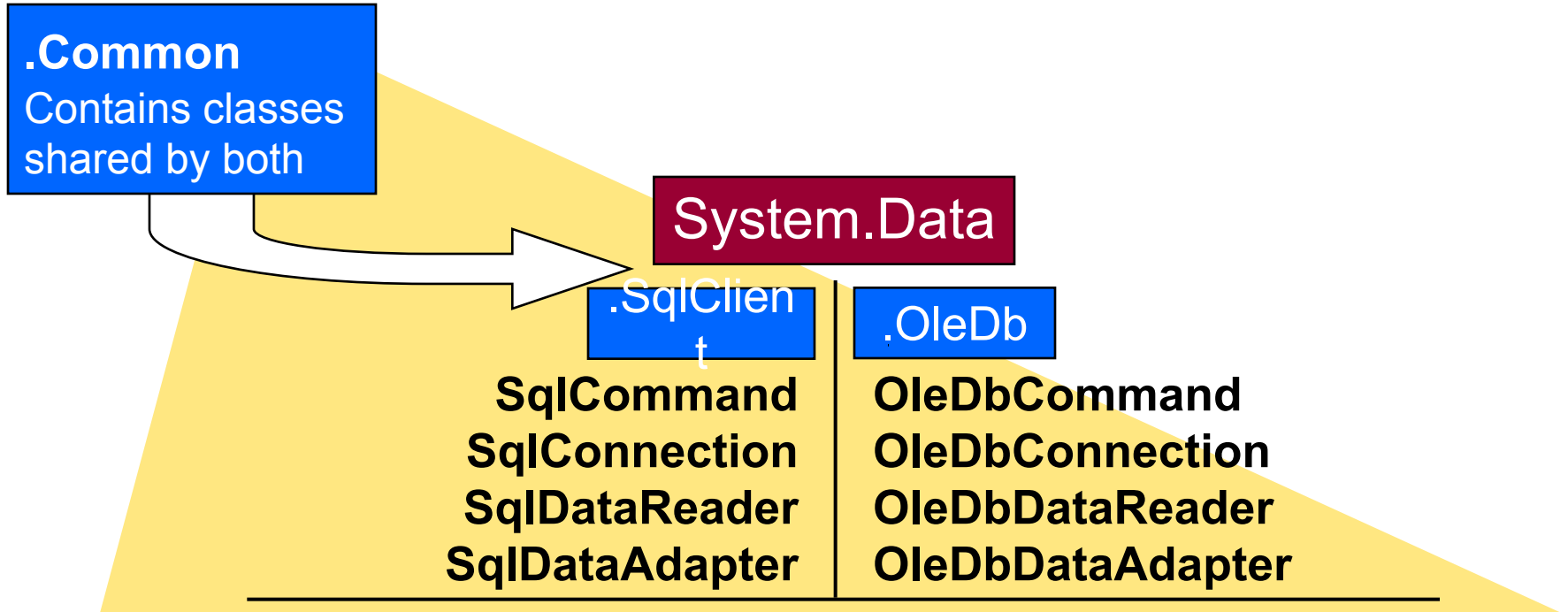


[Class Browser](#) for System.data and System.data.sqlclient

The (ADO).NET Data Providers

- A collection of classes for accessing data sources:
 - Microsoft SQL Server™ 2000, SQL Server 7, and MSDE
 - Any OLE Database (OLE DB) providers
 - Including: Oracle, JET, and SQL OLE DB Providers
 - Establish connection between **DataSets** and data stores
- Two .NET data providers:
 - ADO: via the **System.Data.OleDb** namespace
 - SQL Server: via the **System.Data.SqlClient** namespace
- **System.Data.OleDb** is *the* .NET data provider

.NET Data Providers Hierarchy



General Steps for Using Web Databases

1. Build your database tables and queries
2. Create a connection to the database
 - The connection identifies the location of the database (the data source) and the connection method (an ODBC driver, OLE-DB provider, or an OLE-DB.NET data provider), along with any other settings such as username or password
3. Create an ASP.NET Web page
4. Add an ADO.NET connection object that connects to the database, executes commands, and returns data from the database
5. Create code that will interact with the data, display the data in an ASP.NET control, perform calculations on the data, or upload changes to the database

ADO.Net – Introducing the objects

- Connection
 - used to talk to DB; properties include dataSource, username and password
 - SqlConnection and OleDbConnection
- Command
 - An SQL statement or Stored Procedure
 - SqlCommand and OleDbCommand
- DataReader- read only, forward only view of data
CF ADO Recordset
- DataSet - main object for DB access
- DataView - filtered view of DataSet
- DataAdapter - Initialises DataSet tables

Introducing the Objects cont.

- **Connections.** For connection to and managing transactions against a database.
- **Commands.** For issuing SQL commands against a database.
- **DataReaders.** For reading a forward-only stream of data records from a SQL Server data source.
- **DataSets.** For storing, remoting and programming against flat data, XML data and relational data.
- **DataAdapters.** For pushing data into a **DataSet**, and reconciling data against a database.

Introducing the Objects cont.

System.Data

DataSet

DataTable

DataRow

DataColumn

DataRelation

DataViewManager

- Contains the “main” classes of ADO.NET
- In-memory cache of data
- In-memory cache of a database table
- Used to manipulate a row in a **DataTable**
- Used to define the columns in a **DataTable**
- Used to relate 2 **DataTables** to each other
- Used to create views on **DataSets**

System.Data Namespace Contains the basis and bulk of ADO.NET

OleDbConnection and SqlConnection

- Represent a unique session with a data source
- Create, open, close a connection to a data source
- Functionality and methods to perform transactions
- **OleDbConnection** example:

```
String conStr="Provider=Microsoft.Jet.OLEDB.4.0;" +  
            "Data Source=NWIND_RW.MDB";  
OleDbConnection aConn = new OleDbConnection(conStr);  
aConn.Open();  
    // Execute Queries using OleDbDataAdapter Class  
aConn.Close();
```

Data Connection Properties

- SQL Server Name
 - Default name of the MSDE version of SQL Server is MachineName\NetSDK
 - MachineName is the name of your local computer
 - Also referred to as (local)\NetSDK or localhost
 - Not required in the Connection String – assumed to be SQL Server if it uses the SQLClient class

Dataset object

- **DataSet** object represents a cache of data, with database-like structures such as tables, columns, relationships, and constraints.
- **DataSet** can and does behave much like a database, it is important to remember that **DataSet** objects do not interact directly with databases, or other source data.
- Allows the developer to work with a programming model that is always consistent, regardless of where the source data resides. Data coming from a database, an XML file, from code, or user input can all be placed into **DataSet** objects.
- Changes made to the **DataSet** can be tracked and verified before updating the source data. The **GetChanges** method of the **DataSet** object actually creates a second **DataSet** that contains only the changes to the data. This **DataSet** is then used by a **DataAdapter** (or other objects) to update the original data source.
- For long-running applications this is often the best approach.

DataAdapter

- To perform a select query to a SQL database, you create a **SqlConnection** to the database passing the connection string, and then construct a **SqlDataAdapter** object that contains your query statement. To populate a **DataSet** object with the results from the query, you call the command's **Fill** method.

```
Dim myConnection As New  
    SqlConnection("server=(local)\NetSDK;database=pubs;Truste  
d_Connection=yes")
```

```
Dim myCommand As New SqlDataAdapter("select * from  
    Authors", myConnection)
```

```
Dim ds As New DataSet() myCommand.Fill(ds, "Authors")
```

DataReader Object

- For Web applications, you are usually performing short operations with each request (commonly to simply display the data). You often don't need to hold a **DataSet** object over a series of several requests. For situations like these, you can use a **SqlDataReader**.
- A **SqlDataReader** provides a forward-only, read-only pointer over data retrieved from a SQL database.
- To use a **SqlDataReader**, you declare a **SqlCommand** instead of a **SqlDataAdapter**.
- The **SqlCommand** exposes an **ExecuteReader** method that returns a **SqlDataReader**.
- Note also that you must explicitly open and close the **SqlConnection** when you use a **SqlCommand**. After a call to **ExecuteReader**, the **SqlDataReader** can be bound to an ASP.NET server control.

DataReader cont.

```
Dim myConnection As SqlConnection = New
    SqlConnection("server=(local)\NetSDK;database=pubs;Trusted_Connection=yes")
Dim myCommand As SqlCommand = New SqlCommand("select * from Authors",
    myConnection)
myConnection.Open()
Dim dr As SqlDataReader = myCommand.ExecuteReader() ...
myConnection.Close()
```

- When performing commands that do not require data to be returned, such as inserts, updates, and deletes, you also use a **SqlCommand**.
- The command is issued by calling an **ExecuteNonQuery** method, which returns the number of rows affected. Note that the connection must be explicitly opened when you use the **SqlCommand** (the **SqlDataAdapter** automatically handles opening the connection for you).

```
Dim myConnection As New
    SqlConnection("server=(local)\NetSDK;database=pubs;Trusted_Connection=yes")
Dim myCommand As New SqlCommand( "UPDATE Authors SET phone='(800)
    555-5555' WHERE au_id = '123-45-6789'", myConnection)
myCommand.Connection.Open()
myCommand.ExecuteNonQuery()
myCommand.Connection.Close()
```

DB Connection Example-1

```
<%@ Page Language="vb" %>
```

```
<%@ Import Namespace="System.Data" %>
```

```
<%@ Import Namespace="System.Data.SqlClient" %>
```

```
<HTML>
```

```
<body>
```

DB Connection Example-2

```
<SCRIPT language="VB" runat="Server">  
Sub Page_Load(Src As Object, E As EventArgs)  
    Dim ds As DataSet  
    Dim conn As SqlConnection  
    Dim cmdAuthors As SqlDataAdapter  
    Dim dv As DataView  
    'create a connection to the Pubs database'  
    conn = New SqlConnection _  
        ("server=localhost;uid=sa;pwd=super;database=pubs")  
    'create a dataset with information from the authors table'  
    cmdAuthors = New SqlDataAdapter _  
        ("select * from Authors", conn)  
    ds = new DataSet()  
    cmdAuthors.Fill(ds, "Authors") 'Authors is the DataTable name in ds
```

DB Connection Example-3

'bind the first datagrid to the DefaultView of the dataset'

```
dv = ds.Tables("Authors").DefaultView
```

```
dgAuthors.DataSource = dv
```

```
dgAuthors.DataBind()
```

'create a new DataView that is authors from California'

'and bind the second datagrid to it'

```
dv = New DataView(ds.Tables("Authors"))
```

```
dv.RowFilter = "state = 'CA'"
```

```
dgCAAuthors.DataSource = dv
```

```
dgCAAuthors.DataBind()
```

```
End Sub
```

```
</SCRIPT>
```

DB Connection Example-4

<h2>All Authors</H2>

<ASP:DataGrid id="dgAuthors" runat="server"

Width="700"

BackColor="#ccccff"

BorderColor="black"

ShowFooter="false"

CellPadding=3

CellSpacing="0"

Font-Name="Verdana"

Font-Size="8pt"

HeaderStyle-BackColor="#aaaadd"/>

<H2>California Authors</H2>

<ASP:DataGrid id="dgCAAuthors" runat="server" />

</body>

</html>

DB Connection Example-5

- Demo the previous code.
- <http://interdev.csse.monash.edu.au/cse2030/jason1/grid.aspx>
- NOTE:
 - Namespaces included in page directives
 - Objects used:
 - SqlConnection ; SqlDataAdapter; Dataset; DataView;
 - Web Form Controls used:
 - <ASP:DataGrid>
 - Grid.DataBind() moves data from memory (dataview) to web page
 - DataGrid does not have to be bound to dataset; can be bound to a hashtable say
 - <http://jasonc.csse.monash.edu.au/chapter7/datagridsimple.aspx>
 - For source see <http://www.csse.monash.edu.au/courseware/cse2030/2002/datagridsimple.txt>

Working Data - The DataSet

- An in-memory cache of data from a data source
- Common way to represent and manipulate data
 - Universal data container
 - Not just for use with databases
- Logical *or* physical representation of data
- Designed to be disconnected from the data source
 - Connect, execute query, disconnect
- Can use XML
 - To read and write data
 - To read and write XMLSchema

Properties & Methods of Interest

- Collections are used to add & remove tables & relations
- Properties of Interest:
 - **Tables**: Returns the collection of **DataTable** objects
 - **Relations**: Returns the collection of **DataRelations**
 - **Namespace**: Gets or sets the namespace of the **DataSet**
- Using Properties Samples:
 - `myDataSet.Tables.Add( myTable );`
 - `myDataTableCollection = myDataSet.Tables`

The DataTable

- May be mapped to a physical table in the data source
- Can be related to one another through **DataRelations**
- Optimistic concurrency or locking - model
- Properties of Interest:
 - **Columns**: Returns **ColumnsCollection** of **DataColumns**
 - **Rows**: Returns **DataRow** objects as a **RowsCollection**
 - **ParentRelations**: Returns the **RelationsCollection**
 - **Constraints**: Returns the table's **ConstraintsCollection**
 - **DataSet**: Returns the **DataSet** of the **DataTable**
 - **PrimaryKey**: Gets the **DataColumns** that make up the table's primary key

System.Data—DataSet and DataTable

- Create a **DataTable** and add it to a **DataSet**

```
DataSet ds = new DataSet();  
  
// Create DataTable object: “Customers”.  
DataTable dt= new DataTable( “Customers” );  
  
// Create and add columns to the table  
// 1. Explicitly create and Add a DataColumn  
DataColumn dc;  
dc = new DataColumn( “CustID”, Type.GetType("System.Int16"));  
dt.Columns.Add( dc );  
  
// 2. Implicitly Create and Add columns (DataColumn).  
dt.Columns.Add( “First_Name”,Type.GetType("System String"));  
dt.Columns.Add( “Last_Name”, Type.GetType("System String"));  
  
// Add the DataTable object to the DataSet  
ds.Tables.Add( dt );
```

Viewing Data - The DataView

- Create multiple views on **DataTable** objects
- Bindable to user interface controls
- Properties of Interest:
 - **Table**: Retrieves or sets the associated **DataTable**
 - **Sort**: Gets or sets the table's sort columns and sort order
 - **RowFilter**: Gets or sets the expression used to filter rows
 - **RowStateFilter**: Gets or sets the row state filter
 - **None**, **Unchanged**, **New**, **Deleted**, **ModifiedCurrent**, and others

Creating a DataView by Example

```
// Code for myTable “Customers” with “Name” column not shown  
DataView view1 = new DataView( myTable );  
DataView view2 = new DataView( myTable );  
  
// Creates Ascending view of Customers by “Name”  
view1.Sort = “Name ASC”;  
  
// Set the view to show only modified (original) rows  
view2.RowStateFilter= DataViewRowState.ModifiedOriginal;  
  
// Bind to UI element(s)...  
DataGrid myGrid = new DataGrid();  
myGrid.SetDataBinding( view1, “Customer”);  
  
//...
```

Relational Databases

Stored Procedure Example

```
CREATE PROCEDURE CustOrderHist @CustomerID nchar(5)
AS
SELECT ProductName, Total=SUM(Quantity)
FROM Products P, [Order Details] OD,
     Orders O, Customers C
WHERE C.CustomerID = @CustomerID
AND C.CustomerID = O.CustomerID
AND O.OrderID = OD.OrderID
AND OD.ProductID = P.ProductID
GROUP BY ProductName
```


ADO.NET - Data Binding

- Key component of Web Forms framework
- Flexible and easy to use
 - Bind a control's property to information in any type of data store
 - Provides control over how data moves back and forth
 - Simple controls for displaying a single value eg below using binding tags <%# %>
 - Complex controls for displaying a data structure eg datagrid

```
<asp:Label id="SelectedValue"runat=server  
Text='<%# lstLocation.SelectedItem.Text %>' />
```

ADO.NET Classes

DataSet Example

```
string sConnString = "Persist Security Info=False;" +  
    "User ID=sa;Initial Catalog=Northwind;" +  
    "Data Source=MYSERVER";  
SqlConnection conn = new SqlConnection(sConnString);  
conn.Open();  
string sQueryString = "SELECT CompanyName FROM Customers";  
SqlDataAdapter myDSAdapter = new SqlDataAdapter();  
DataSet myDataSet = new DataSet();  
myDSAdapter.SelectCommand = new SqlCommand(sQueryString, conn);  
myDSAdapter.Fill(myDataSet);  
conn.Close();
```

Stored Procedure Example

```
<%@ Page Language="vb" %>
```

```
<%@ Import Namespace="System.Data" %>
```

```
<%@ Import Namespace="System.Data.SqlClient" %>
```

```
<HTML> <body>
```

```
<SCRIPT language="VB" runat="Server">
```

```
Dim conn as SqlConnection
```

```
Sub Page_Load(Src As Object, E As EventArgs)
```

```
    conn = New  
    SqlConnection("server=localhost;uid=sa;pwd=1Aspt;database=conf")
```

```
    displayCategories()
```

```
    displayProducts()
```

```
    displayOrderCount()
```

```
End Sub
```

'the ProductCategoryList storedprocedure has no parameters and returns 'records. display the returned records in a datagrid'

Sub displayCategories()

Dim cmd As SQLDataAdapter

Dim ds As DataSet

Dim workParam As SqlParameter = Nothing

'call the ProductCategoryList stored procedure'

cmd = New SQLDataAdapter("ProductCategoryList", conn)

'fill dataset with results of stored procedure'

ds = new DataSet()

cmd.Fill(ds, "Categories")

'bind dataset to datagrid'

dgCategories.DataSource = ds.Tables("Categories").DefaultView

dgCategories.DataBind()

End Sub

'the ProductsByCategory storedprocedure has an input parameter which is the categoryID'
'and returns all items from that category'
'read the input parameter from a text box and display the results in a datagrid'

Sub displayProducts()

Dim cmd As SqlDataAdapter

Dim ds As DataSet

Dim workParam As SqlParameter = Nothing

'call the ProductCategory stored procedure'

cmd = New SqlDataAdapter("ProductsByCategory", conn)

cmd.SelectCommand.CommandType = CommandType.StoredProcedure

'add the CategoryID input parameter from the txtCatID textbox'

workParam = New SqlParameter("@CategoryID", SqlDbType.Int)

workParam.Direction = ParameterDirection.Input

workParam.Value = CInt(txtCatID.Text)

cmd.SelectCommand.Parameters.Add (workParam)

'run the stored procedure and fill a dataset with the results'

ds = new DataSet()

cmd.Fill(ds, "Products")

'bind the dataset to a datagrid'

dgProducts.DataSource = ds.Tables("Products").DefaultView

dgProducts.DataBind()

³⁷
End Sub

'the OrdersCount storedprocedure has an input parameter which is the customerID' 'and an output parameter which is the number of orders for that customer.'"read the input parameter from a text box and display the output value in a label'

```
Sub displayOrderCount()  
    Dim cmd As SqlCommand  
    Dim workParam As SqlParameter = Nothing  
  
    'call OrdersCount stored procedure'  
    cmd = New SqlCommand()  
    With cmd  
        .Connection = conn  
        .CommandText = "OrdersCount"  
        .CommandType = CommandType.StoredProcedure  
    End With
```

'add the CustomerID input parameter from txtCustID textbox'

```
workParam = New SqlParameter("@CustomerID", SqlDbType.Int)  
workParam.Direction = ParameterDirection.Input  
workParam.Value = CInt(txtCustID.Text)  
cmd.Parameters.Add(workParam)
```

'add the ItemCount output parameter'

```
workParam = New SqlParameter("@ItemCount", SqlDbType.Int)  
workParam.Direction = ParameterDirection.Output  
cmd.Parameters.Add(workParam)
```

'open the connection so you can call execute on the SelectCommand'

```
conn.Open()  
cmd.ExecuteNonQuery()  
conn.Close()
```

'display the output parameter in a SPAN element'

```
spnOrderCount.InnerHTML =  
cmd.Parameters("@ItemCount").Value  
End Sub
```

</SCRIPT>

```
<h2>Categories</h2>
```

```
<asp:datagrid id="dgCategories" runat="server"/>
```

```
<br><br>
```

```
<form runat="server">
```

```
<P>Enter category: <asp:textbox id="txtCatID" runat="server"
Text="14"/>
```

```
<asp:button runat="server" text="Get Products"/>
```

```
<h2>Products in Category</h2>
```

```
<P><asp:datagrid id="dgProducts" runat="server"/>
```

```
<br><br>
```

```
<h2>Number of Current Orders for a Customer</h2>
```

```
<P>Customer ID <asp:textbox id="txtCustID" runat="server"
Text="31"/>
```

```
<asp:button runat="server" text="Get Order Count"/>
```

```
<br>has <span id="spnOrderCount" runat="server"></span> outstanding
order(s)
```

```
</form> </body> </html>
```


Demo Stored Procedure example

- <democode/mod03/storedprocedure.aspx>
- Repeater Example
 - [Simple List -repeater Example](#)
 - Uses templates for formatting output...to be discussed next lecture

Accessing XML-based Data

- The **DataSet** was designed to abstract data in a way that is independent of the actual data source.
- Change the focus of your samples from SQL to XML. The **DataSet** supports a **ReadXml** method that takes a **FileStream** object as its parameter.
- The file you read in this case must contain both a schema and the data you wish to read.
- Datagrid example17 – [read XML data](#)

DataGrid control – some examples

- [Datagrid properties](#)
- Updating Data in a SQL Database -
gatagrid6.aspx to datagrid10.aspx